

From Business Tools to AI-Ready Tools: An MCP Build Log

I spent the last week turning a set of internal business tools into something an AI model can actually operate — and it taught me more about production AI integration than any tutorial has.

The starting point: a Python codebase with tools an LLM was already calling directly — customer lookup, product search, bill creation, the usual business-API surface. Useful, but tightly coupled to one setup, with no clean way for a different AI client to plug into it.

The goal: rebuild it as an MCP (Model Context Protocol) server — the open standard that lets any AI client (Claude, and increasingly others) discover and call a set of tools the same structured way, instead of every integration being bespoke.

A few things stood out along the way:

- A tool's docstring isn't documentation, it's the interface. The model reads it to decide when to call your function. A vague one means the wrong tool gets picked, or the right one doesn't.
- Type hints ARE the contract. The SDK builds validation straight from them — meaning malformed input gets rejected automatically, before your code even runs.
- Error handling has to be deliberate. There's a real difference between "the model gave bad input and can fix it" and "the backend is down and no amount of retrying helps" — treating both the same makes for a much worse AI experience.
- Some of the best bugs were the most boring ones. A backend function's signature changed during a refactor, but the caller wasn't updated — four working call sites became a plain old Python `TypeError`. AI tooling doesn't remove the need for basic engineering discipline, it just moves where it bites you.
- Debugging a protocol server isn't like debugging a normal script — `stdout` is literally the wire, so your usual `print` statements silently vanish. Small, unglamorous lessons like this are where the real learning lives.

End result: the same business logic I'd already built, now callable by Claude directly through a real conversation — search a customer, create a bill, all through natural language, backed by actual validated, testable tool code underneath.

MCP is still early, but this is clearly where a lot of practical AI integration work is headed — not bigger prompts, but well-defined, well-typed tools an AI can reliably operate.

Happy to go deeper with anyone building something similar, or share what the actual server code looks like.

[#MCP](#) [#AIEngineering](#) [#SoftwareDevelopment](#) [#Python](#) [#ModelContextProtocol](#)